

Complete Interview-Level Guide:

Configuring ESLint, Prettier & Husky with Pre-Commit Hooks

1. Project Context & Objective

In our full-stack project (Node.js backend + React frontend), we implemented ESLint, Prettier, and Husky to ensure code quality, formatting consistency, and automated pre-commit validation.

Objective:

- Maintain clean and consistent code.
- Prevent bad code from entering the repository.
- Reduce code review effort.
- Enforce team-wide coding standards.

2. What is ESLint? (Deep Understanding)

ESLint is a static code analysis tool for identifying problematic patterns in JavaScript code.

Technical Role:

- Detect unused variables.
- Enforce best practices.
- Prevent syntax mistakes.
- Maintain coding standards.

ESLint works by parsing JavaScript into an AST (Abstract Syntax Tree) and applying rule-based validations.

3. ESLint Installation & Setup (Backend Example)

Step 1: Install ESLint `npm install eslint --save-dev` Step 2: Initialize configuration `npx eslint --init`

During setup we selected:

- Environment: Node.js
- Module Type: CommonJS/ESM
- Style Guide: Recommended
- Config format: JavaScript

Step 3: Create `eslint.config.js` Configured with recommended rules and Node environment.

4. ESLint Configuration Logic

Key Configurations:

- Use recommended rules.
- Ignore `node_modules`.
- Define environment as `node`.
- Integrate Prettier to avoid formatting conflicts.

Example Script in `package.json`:

```
"scripts": {  
  "lint": "eslint .",  
  "lint:fix": "eslint . --fix"  
}
```

lint: checks errors
lint:fix: auto-fixes fixable issues

5. What is Prettier? (Advanced Explanation)

Prettier is an opinionated code formatter. Difference from ESLint: • ESLint = Finds logical/code quality issues • Prettier = Handles formatting only Prettier ensures: • Consistent indentation • Quote style • Semicolons • Line wrapping

6. Prettier Setup

Install: `npm install --save-dev prettier eslint-config-prettier eslint-plugin-prettier` Create `.prettierrc`: { "semi": true, "singleQuote": true, "tabWidth": 2, "trailingComma": "es5" } `eslint-config-prettier` disables ESLint rules that conflict with Prettier. `eslint-plugin-prettier` runs Prettier as an ESLint rule.

7. Why Husky is Important (Professional Workflow)

Husky enables Git hooks. Problem: Developers may forget to run lint manually before commit. Solution: Use pre-commit hook to automatically run lint before every commit. If lint fails → commit is blocked.

8. Husky Setup in Project

Install Husky: `npm install husky --save-dev` Initialize: `npx husky init` Add pre-commit hook: `npx husky add .husky/pre-commit "npm run lint"` Now before every commit: • ESLint runs automatically. • If errors exist → commit fails. • If clean → commit succeeds.

9. Team Workflow (Dev1 & Dev2 Scenario)

Workflow Used: • Dev1 works on feature branch. • Dev2 works on another branch. • Each developer commits code. • Husky ensures lint passes locally. • Pull Request is created to develop branch. • Clean code reduces merge conflicts. This ensures production-level discipline.

10. Interview-Level Explanation (How to Answer)

“In our project, we implemented ESLint for static analysis and Prettier for formatting consistency. To enforce these rules automatically, we integrated Husky pre-commit hooks, which run lint before every commit. This prevents bad-quality code from entering the repository and ensures consistent coding standards across the team.” Advanced Add-On: You can also mention CI/CD pipeline integration where lint runs in GitHub Actions.

11. Advanced Improvements (Professional Level)

For even better workflow, we can add:

- lint-staged → Runs lint only on staged files.
- CI/CD integration → Run lint on every push.
- Commitlint → Enforce commit message standards.
- Conventional commits strategy.

12. Benefits of This Setup

- Automated quality control
- Clean Git history
- Reduced code review time
- Better team collaboration
- Production-ready standards
